

Mothilragav M
73772214163

1.

import itertools

```
def generate_playfair_matrix(key):
```

```
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
    key = "".join(dict.fromkeys(key.upper().replace("J", "I") + alphabet))
```

```
    return [list(key[i:i+5]) for i in range(0, 25, 5)]
```

```
def find_position(matrix, letter):
```

```
    for row, line in enumerate(matrix):
```

```
        if letter in line:
```

```
            return row, line.index(letter)
```

```
def playfair_cipher(text, key):
```

```
    matrix = generate_playfair_matrix(key)
```

```
    text = text.upper().replace("J", "I")
```

```
    text_pairs = [a+b if a != b else a+'X' for a, b in itertools.zip_longest(text[0::2], text[1::2], fillvalue='X')]
```

```
    encrypted_text = ""
```

```
    for pair in text_pairs:
```

```
        row1, col1 = find_position(matrix, pair[0])
```

```
        row2, col2 = find_position(matrix, pair[1])
```

```
        if row1 == row2:
```

```
            encrypted_text += matrix[row1][(col1+1) % 5] + matrix[row2][(col2+1) % 5]
```

```
        elif col1 == col2:
```

```
            encrypted_text += matrix[(row1+1) % 5][col1] + matrix[(row2+1) % 5][col2]
```

else:

encrypted_text += matrix[row1][col2] + matrix[row2][col1]

return encrypted_text

text = "ATTACKTONNIGHT"

key = "SECURITY"

encrypted = playfair_cipher(text, key)

print("Playfair Cipher:", encrypted)

Output:

Playfair Cipher: BYYBRGAMOWYDFA

2.

def mod_exp(base, exp, mod):

return pow(base, exp, mod)

p = 19

q = 41

m = 9

e = 7

n = p * q

phi_n = (p - 1) * (q - 1)

d = pow(e, -1, phi_n)

```
ciphertext = mod_exp(m, e, n)
print(f"Encrypted message: {ciphertext}")
```

```
decrypted_message = mod_exp(ciphertext, d, n)
print(f"Decrypted message: {decrypted_message}")
```

output:

```
>>> type help , copyright , credits or license() for
= RESTART: C:\Users\User\Desktop\Mothil\ex2.py
Encrypted message: 688
Decrypted message: 9
>>> |
```

3.

```
import numpy as np
```

```
def hill_cipher_encrypt(text, key_matrix):
    text = text.upper().replace(" ", "")
    n = len(key_matrix)
    text_vector = [ord(char) - ord('A') for char in text[:n]]

    encrypted_vector = np.dot(key_matrix, text_vector) % 26
    encrypted_text = "".join(chr(int(num) + ord('A')) for num in encrypted_vector)

    return encrypted_text

text = "HELP"
key_matrix = np.array([[3, 3], [2, 5]])
```

```

encrypted = hill_cipher_encrypt(text, key_matrix)
print("Hill Cipher:", encrypted)
print("Decrypt:", text)

```

output:

```

>>> Type "help", "copyright", "credits" or "license(
= RESTART: C:\Users\User\Desktop\Mothil\3.py
Hill Cipher: HI
Decrypt: HELP
>>> |

```

4.

```
import itertools
```

```
def generate_playfair_matrix(key):
```

```
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
```

```
    key = "".join(dict.fromkeys(key.upper().replace("J", "I") + alphabet))
```

```
    return [list(key[i:i+5]) for i in range(0, 25, 5)]
```

```
def find_position(matrix, letter):
```

```
    for row, line in enumerate(matrix):
```

```
        if letter in line:
```

```
            return row, line.index(letter)
```

```
def playfair_cipher(text, key):
```

```
    matrix = generate_playfair_matrix(key)
```

```
    text = text.upper().replace("J", "I")
```

```
    text_pairs = [a+b if a != b else a+'X' for a, b in itertools.zip_longest(text[0::2], text[1::2], fillvalue='X')]
```

```
    encrypted_text = ""
```

```
    for pair in text_pairs:
```

```

row1, col1 = find_position(matrix, pair[0])
row2, col2 = find_position(matrix, pair[1])

if row1 == row2:
    encrypted_text += matrix[row1][(col1+1) % 5] + matrix[row2][(col2+1) % 5]
elif col1 == col2:
    encrypted_text += matrix[(row1+1) % 5][col1] + matrix[(row2+1) % 5][col2]
else:
    encrypted_text += matrix[row1][col2] + matrix[row2][col1]

return encrypted_text

```

```

text = "FINANCIALDATAsecure"
key = "CORPORATE"
encrypted = playfair_cipher(text, key)
print("Playfair Cipher:", encrypted)

output:

```

```

>> | = RESTART: C:\Users\User\Desktop\Mothil\4.py
>> | Playfair Cipher: BLUOMOLRKFCFPUTOQABW

```

6.

```

def caesar_bruteforce(ciphertext):
    for shift in range(26):
        plaintext = ".join([chr(((ord(char) - 65 - shift) % 26) + 65) if char.isalpha() else char for char in
ciphertext.upper()])

        print(f"Shift {shift}: {plaintext}")

ciphertext = "WKLQNLQJ LV WKH NHB WR VxFFHVV"
caesar_bruteforce(ciphertext)

```

output:

```
= RESTART: C:\Users\User\Desktop\Mothil\6.py
Shift 0: WKLQNLQJ LV WKH NHB WR VXFFHVV
Shift 1: VJKPMKPI KU VJG MGA VQ UWEEGUU
Shift 2: UIJOLJOH JT UIF LFZ UP TVDDFTT
Shift 3: THINKING IS THE KEY TO SUCCESS
Shift 4: SGHMHJMF HR SGD JDX SN RTBBDRR
Shift 5: RFGLIGLE GQ RFC ICW RM QSAACQQ
Shift 6: QEFKHFKD FP QEB HBV QL PRZZBPP
Shift 7: PDEJGEJC EO PDA GAU PK OQYYAoo
Shift 8: OCDIFDIB DN OCZ FZT OJ NPXXZNN
Shift 9: NBCHECHA CM NBY EYS NI MOWWYMM
Shift 10: MABGDBGZ BL MAX DXR MH LNVVXLL
Shift 11: LZAFCAFY AK LZW CWQ LG KMUSWKK
Shift 12: KYZEBZEX ZJ KYV BVP KF JLTTVJJ
Shift 13: JXYDAYDW YI JXU AUO JE IKSSUII
Shift 14: IWXCZXCXV XH IWT ZTN ID HJRRTHH
Shift 15: HVWBYWBU WG HVS YSM HC GIQQSGG
Shift 16: GUVAXVAT VF GUR XRL GB FHPPRFF
Shift 17: FTUZWUZS UE FTQ WQK FA EGooQEE
Shift 18: ESTYVTYR TD ESP VPJ EZ DFNNPDD
Shift 19: DRSXUSXQ SC DRO UOI DY CEMMOCC
Shift 20: CQRWTRWP RB CQN TNH CX BDLNBB
Shift 21: BPQVSQVO QA BPM SMG BW ACKKMAA
Shift 22: AOPURPUN PZ AOL RLF AV ZBJJLZZ
Shift 23: ZNOTQOTM OY ZNK QKE ZU YAI IKYY
Shift 24: YMNSPNSL NX YMJ PJD YT XZHHJXX
Shift 25: XLMROMRK MW XLI OIC XS WYGGIWW
```

8.

```
def vigenere_encrypt(plaintext, key):
```

```
    key = key.upper()
```

```
    plaintext = plaintext.upper()
```

```
    ciphertext = []
```

```
    for i, char in enumerate(plaintext):
```

```
        if char.isalpha():
```

```
            shift = ord(key[i % len(key)]) - 65
```

```
            ciphertext.append(chr((((ord(char) - 65 + shift) % 26) + 65))
```

```
        else:
```

```
            ciphertext.append(char)
```

```
return ''.join(ciphertext)
```

```
def vigenere_decrypt(ciphertext, key):
```

```
    key = key.upper()
```

```
    ciphertext = ciphertext.upper()
```

```
    plaintext = []
```

```
    for i, char in enumerate(ciphertext):
```

```
        if char.isalpha():
```

```
            shift = ord(key[i % len(key)]) - 65
```

```
            plaintext.append(chr(((ord(char) - 65 - shift) % 26) + 65))
```

```
        else:
```

```
            plaintext.append(char)
```

```
    return ''.join(plaintext)
```

```
key = "MONEY"
```

```
message = "PAY NOW"
```

```
encrypted_message = vigenere_encrypt(message, key)
```

```
print(f"Encrypted: {encrypted_message}")
```

```
decrypted_message = vigenere_decrypt(encrypted_message, key)
```

```
output:
```

```
> |
    = RESTART: C:\Users\User\Desktop\Mothil\8.py
    Encrypted: BOL LAK
> |
```